Autonomous Single-Anchor Homing Navigation for Quadrotors: A Comprehensive Analysis of Range-Only Control Strategies

1. Introduction

The pursuit of autonomous navigation for Unmanned Aerial Vehicles (UAVs) in Global Navigation Satellite System (GNSS)-denied environments represents a significant frontier in modern robotics research. While complex solutions involving Simultaneous Localization and Mapping (SLAM) utilizing LiDAR or stereo vision are prevalent, they often impose substantial computational and payload burdens on small-scale platforms. A minimalist alternative, increasingly relevant for swarm robotics and resource-constrained nano-UAVs, is range-only navigation. This paradigm restricts the robotic agent's sensory input to a single scalar value: the Euclidean distance to a fixed radio beacon or "anchor." This scenario effectively transforms the navigation challenge into a source-seeking problem within a scalar potential field, devoid of instantaneous bearing information.

This report provides an exhaustive technical analysis of the algorithms and control logic required to enable a Bitcraze Crazyflie drone to autonomously navigate toward a single fixed anchor using Ultra-Wideband (UWB) range measurements. The specific system architecture involves a Crazyflie 2.X equipped with a Loco Positioning Deck utilizing the Decawave DW1000 transceiver, performing Single-Sided Two-Way Ranging (SS-TWR). The control loop is closed off-board via a host computer using the Python cflib library, with a sampling interval of 20 milliseconds (50 Hz). This configuration presents a unique set of constraints—specifically network latency, measurement noise, and the fundamental unobservability of bearing from a single static measurement—that necessitates sophisticated temporal processing and active sensing strategies.

The analysis that follows delves into the theoretical underpinnings of scalar field guidance, the stochastic characterization of UWB signals, and the derivation of three distinct control architectures: Reactive Gradient Following, Extremum Seeking Control (ESC), and Temporal Trilateration. Each strategy is evaluated against the specific dynamics of the Crazyflie platform and the limitations of the Python-based communication link. The objective is to provide a robust, theoretically sound, and practically implementable roadmap for achieving stable homing behavior without reliance on external motion capture systems or optical flow integration.

2. Theoretical Fundamentals of Range-Only Guidance

2.1 The Geometry of Underspecified Localization

The fundamental difficulty in single-anchor navigation lies in the geometry of the measurement. A single range measurement r defines a solution space corresponding to the surface of a sphere (in 3D) or a circle (in 2D, assuming constant altitude) centered at the anchor position $\mathbf{p}_a = [x_a, y_a, z_a]^T$. For a drone at position $\mathbf{p} = [x, y, z]^T$, the measurement function is given by the nonlinear equation:

$$h(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_a\| = \sqrt{(x-x_a)^2 + (y-y_a)^2 + (z-z_a)^2} + \eta$$

where η represents measurement noise.[1] Unlike vector sensors such as cameras or directional antennas that provide a bearing vector $\mathbf{u}_{bearing}$ allowing for immediate course correction, the range sensor provides no information regarding the direction of the gradient $\nabla h(\mathbf{p})$. The gradient vector, which points directly away from the anchor, is the critical piece of information required for homing.

$$\nabla h(\mathbf{p}) = \frac{\mathbf{p} - \mathbf{p}_a}{\|\mathbf{p} - \mathbf{p}_a\|}$$

The control objective is to align the drone's velocity vector $\mathbf{v}$ with the negative gradient $-\nabla h(\mathbf{p})$. However, because the drone cannot measure $\mathbf{p}$ or $\mathbf{p}_a$ directly, $\nabla h(\mathbf{p})$ is essentially unobservable from a single snapshot. This leads to the concept of *observability through motion*. For the system to resolve the bearing of the anchor, it must change its state (position) and observe the resulting change in the scalar measurement. This transforms the spatial problem into a spatio-temporal one, where the controller must essentially perform an online system identification task to estimate the relative bearing based on the correlation between control inputs (velocity/yaw commands) and sensor outputs (range rates).[3]

2.2 The Scalar Field Source Seeking Problem

The homing task can be rigorously formulated as a generic source-seeking problem, where the agent seeks the minimum of a scalar cost function $J(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_a\|$. In classical optimization theory, searching for a minimum without gradient information requires numerical approximation methods. The drone effectively acts as a physical optimizer, exploring the cost landscape. The "cost" surface is a convex bowl (a cone, precisely) with the global minimum at the anchor.

However, theoretical analysis typically assumes an ideal quadratic or convex field. In reality, the UWB signal strength and range quality can be affected by the antenna radiation pattern, which is not perfectly isotropic.[5] The Crazyflie's PCB antenna exhibits gain variations depending on orientation, and the UWB signal itself is subject to multipath interference,

creating local minima or "phantom" gradients in the scalar field.[6] Therefore, the navigation algorithm must be robust not just to Gaussian noise but to non-convexities in the perceived potential field.

2.3 System Dynamics and the Unicycle Model

For the purpose of designing the guidance law, the complex quadrotor dynamics can be abstracted. The inner-loop attitude controller (running on the STM32 firmware) handles the high-frequency stabilization, allowing the Python outer-loop to treat the drone as a kinematic particle or a unicycle. The "Unicycle Model" is particularly appropriate for this application because the Crazyflie, while capable of holonomic movement (moving sideways), is often controlled most effectively by coupling yaw with forward flight to point sensors or directional antennas.[4]

The equations of motion for the guidance layer are:

$$\begin{aligned} \dot{x} &= v \cos(\psi) \\ \dot{y} &= v \sin(\psi) \\ \dot{\psi} &= \omega \end{aligned}$$
where v is the forward velocity, ψ is the heading (yaw) angle, and ω is the yaw rate input. The distance to the source is $r = \sqrt{x^2 + y^2}$. Differentiating r with respect to time yields:

$$\dot{r} = \frac{x\dot{x} + y\dot{y}}{r} = v \frac{x \cos \psi + y \sin \psi}{r} = v \cos(\psi - \theta_{anchor})$$
Here, θ_{anchor} is the true bearing to the anchor. This equation reveals the fundamental coupling: the rate of change of range $\dot{r}$ depends on the alignment error $(\psi - \theta_{anchor})$.

- If $\psi = \theta_{anchor}$ (flying away), $\dot{r} = v$ (maximum positive).
- If $\psi = \theta_{anchor} + \pi$ (flying towards), $\dot{r} = -v$ (maximum negative).
- If $\psi = \theta_{anchor} \pm \pi/2$ (orbiting), $\dot{r} = 0$.

This relationship is the cornerstone of all range-only homing algorithms. By manipulating ψ (via ω) and observing $\dot{r}$, the controller can deduce the alignment error and correct it. The 20ms update rate provided by the user is sufficient to capture these dynamics, provided the flight speed v is chosen such that the displacement over a few samples is measurable against the noise floor.[8]

3. Characterization of Ultra-Wideband Ranging Signals

Before any control logic can be applied, the raw input data—the UWB range—must be characterized and conditioned. The user notes the use of the DW1000 chip. While robust,

UWB ranging is not a perfect "truth" sensor; it suffers from distinct error modes that can destabilize feedback loops.

3.1 Anatomy of UWB Measurement Errors

The DW1000 estimates distance based on the Time of Flight (ToF) of radio pulses. The error budget is composed of several distinct components, each requiring a specific mitigation strategy in the software stack.

Table 1: Classification of UWB Ranging Errors

Error Type	Magnitude	Cause	Impact on Homing	Mitigation Strategy
Jitter (Noise)	$\pm 5-10$ cm	Clock drift, thermal noise, timestamping quantization.	Creates "noise" in the gradient estimate $\dot{r}$.	Low-pass or Median Filtering.
Bias (Offset)	$10-30$ cm	Antenna delay calibration errors, frequency dependent attenuation.	Constant offsets do not affect $\dot{r}$, so less critical for homing.	Calibrate anchor delays (optional).
NLOS / Multipath	$0.5 - 2.0$ m	Signal bouncing off walls/floor; direct path blocked by drone body.	Sudden positive jumps in range. Causes "repulsive" force in control.	Outlier rejection, Innovation gating.
Packet Loss	N/A	Interference, antenna nulls, cross-polarization.	Missing data points; variable Δt.	Zero-order hold or predictive filling.

The most dangerous error for homing is the Non-Line-of-Sight (NLOS) or multipath outlier.[6] If

the direct path is blocked (e.g., by the drone's battery or motors during a bank), the radio may lock onto a reflection. This creates a sudden, physically impossible jump in distance. If the controller interprets this as "moving away fast," it might command a violent correction.

3.2 Signal Conditioning Pipeline

Given the 20ms update interval (50 Hz), we have a reasonable temporal resolution to apply digital filters. The raw data stream logged to the computer should pass through a multi-stage conditioning pipeline before entering the guidance logic.

3.2.1 Outlier Rejection via Velocity Gating

The first line of defense is physical feasibility. The Crazyflie has a maximum attainable velocity v_{max}. If the range changes by Δr over time Δt, and $|\Delta r/\Delta t| > v_{max}$, the measurement is likely spurious.
For example, if $v_{max} = 1.0$ m/s and $\Delta t = 0.02$ s, the range cannot change by more than 0.02 m (2 cm) between samples. However, to account for noise, a safety factor is usually applied (e.g., 3σ of the noise + physical limit). A threshold of 0.1 m to 0.2 m per step is reasonable. Spikes exceeding this should be discarded or clamped to the previous valid value.10

3.2.2 The Moving Median Filter

Linear filters like Moving Average (Mean) are sensitive to outliers; a single large spike drags the average resulting in a "smear" that persists for the window duration. Non-linear filters like the Median Filter are superior for UWB data because they completely reject impulsive noise while preserving the sharp edges of true motion.11
A window size of $N=5$ samples is recommended. At 50 Hz, this introduces a latency of roughly $(N-1)/2 \times 20\text{ms} = 40\text{ms}$, which is negligible for the outer guidance loop dynamics.

3.2.3 Gradient Estimation

The control algorithms rely on $\dot{r}$ (range rate). Naive differentiation $(r_k - r_{k-1})/\Delta t$ amplifies the high-frequency quantization noise of the DW1000. A more robust approach is a Savitzky-Golay filter or a linear regression slope calculation over a short window (e.g., 10 samples / 200ms).

$$\dot{r}_{est}(t) = \frac{\sum (t_i - \bar{t})(r_i - \bar{r})}{\sum (t_i - \bar{t})^2}$$

This statistical approach smooths the derivative estimate, providing a cleaner signal for the "Am I getting closer?" decision logic.6

3.3 The Impact of Network Latency

The user is running the logic on a computer. The cflib communication stack introduces non-deterministic latency. A log packet generated at t_0 might arrive at $t_0 +

10\text{ms}$. The command sent back might arrive at the drone at $t_0 + 30\text{ms}$ or later.13
This round-trip delay acts as a phase lag in the control loop. For "slow" guidance (1-2 Hz bandwidth), this is acceptable. However, it necessitates that the drone's inner stabilization loop (on-board) handles the immediate wind disturbances, while the Python script only commands the trend of velocity. The algorithm must be designed to be tolerant of "stale" data—specifically, if a packet is dropped, the Python loop must not treat the repeated previous value as a "zero velocity" condition, but rather handle the irregularity in the time delta dt.

4. Algorithmic Strategy I: Reactive Gradient Following

The simplest and most robust starting point for range-only navigation is a reactive architecture, often referred to in biological contexts as "kinesis" or "biased random walk," and in robotics as "Braitenberg vehicle" logic.[4] This approach requires no internal map, no estimation of the anchor's coordinates, and minimal state memory. It relies purely on the instantaneous sign of the range derivative.

4.1 Concept and Logic

The core principle is an "if-then" behavior tree based on the gradient $\dot{r}$:

1. **Positive Gradient ($\dot{r} > 0$):** The drone is moving *away* from the anchor. Action: **Turn**.
2. **Negative Gradient ($\dot{r} < 0$):** The drone is moving *towards* the anchor. Action: **Go Straight**.
3. **Zero Gradient ($\dot{r} \approx 0$):** The drone is orbiting (tangential). Action: **Turn** (to break symmetry).

This logic causes the drone to persistently turn until it accidentally aligns with the gradient descent direction, at which point it suppresses the turning and accelerates forward. It is essentially a physical implementation of a gradient descent line search.

4.2 Python Implementation Details

For the Crazyflie, this can be implemented using the MotionCommander or low-level Commander to send velocity setpoints. The state machine in the Python loop would look like this:

- **State: SEEKING**
  - Command: $v_x = 0.2$ m/s, $\omega_z = 0.5$ rad/s.
  - Behavior: The drone flies in a circle or spiral.
  - Transition: If $\dot{r} < -\text{threshold}$ (strongly decreasing range), switch to **APPROACH**.
- **State: APPROACH**
  - Command: $v_x = 0.3$ m/s, $\omega_z = 0.0$ rad/s.

    - Behavior: Fly straight forward.
    - Transition: If $\dot{r} > -\text{threshold}$ (range stops decreasing or starts increasing), switch back to **SEEKING**.
- **State: ARRIVAL**
    - Transition: If $r < 0.5$ meters, stop v_x and land or hover.

4.3 Analysis of Behavior

- **Pros:** This method is extremely robust to calibration errors. It does not care if the range is biased by +1 meter, as long as the *change* is consistent. It is also insensitive to magnetic interference affecting the compass, as it works in the body frame.
- **Cons:** The flight path is inefficient. The drone will "zigzag" or spiral towards the target rather than taking a direct path. It also struggles near the anchor, where the relative angular velocity of the LOS vector becomes high, causing the drone to constantly overshoot and circle the target without settling.
- **Suitability:** This is the recommended "Hello World" algorithm for the user to verify their hardware and data link before attempting more complex math.

5. Algorithmic Strategy II: Perturbation-Based Extremum Seeking (ESC)

For a smoother, more efficient, and mathematically elegant solution, Extremum Seeking Control (ESC) is the industry standard for source seeking with nonholonomic vehicles.[9] Unlike the reactive bang-bang controller, ESC continuously estimates the gradient and adjusts the heading in a smooth analog manner.

5.1 Theoretical Formulation

The ESC algorithm treats the drone-anchor system as a "black box" optimization problem where the input is the heading ψ and the output is the cost $J(\psi) = \text{range}$. To find the heading that minimizes range, ESC injects a sinusoidal "dither" signal into the yaw rate and correlates it with the resulting variations in range.

Let the yaw rate control input be:

$$\omega(t) = a \sin(\omega_0 t) + \hat{\omega}_{bias}(t)$$

Here, $a \sin(\omega_0 t)$ is the perturbation (probing signal) and $\hat{\omega}_{bias}$ is the estimated steering correction.
This perturbation causes the drone to "slither" or undulate left and right as it flies forward with constant velocity v.
The Correlation Mechanism:

- If the anchor is to the **left**, turning left (positive yaw phase) causes the range to drop (negative range rate). The product of the dither and the range rate will have a specific DC bias.
- If the anchor is to the **right**, the phase relationship inverts.
- By multiplying (demodulating) the measured signal with the dither signal and integrating the result, the system automatically drives $\hat{\omega}_{bias}$ to steer the drone toward the gradient minimum.

5.2 Mathematical Derivation for Implementation

The specific ESC loop to be implemented in Python involves four distinct stages [18]:

1. Measurement & Washout:
   We measure range $y(t)$. We need the variation, so we apply a High-Pass Filter (HPF) to remove the DC component (absolute distance).

   $$\xi = \frac{s}{s + \omega_h} [y]$$

   In discrete time (Python):
   xi[k] = alpha * xi[k-1] + alpha * (y[k] - y[k-1])
   where alpha depends on the cutoff frequency ω_h. Alternatively, simply using $\dot{r} \approx y_k - y_{k-1}$ serves as a crude high-pass filter (a differentiator).
2. Demodulation:
   Multiply the filtered output ξ by the perturbation carrier signal.

   $$\eta(t) = \xi(t) \cdot \sin(\omega_0 t)$$

   This signal $\eta(t)$ contains a DC component proportional to the gradient ∇J and high-frequency harmonics.
3. Gradient Estimation (Integration):
   Integrate $\eta(t)$ to update the steering bias.

   $$\hat{\omega}_{bias}(t) = -k \int_0^t \eta(\tau) d\tau$$

   Crucial Note: The sign of the gain k determines if we seek a minimum or maximum. For homing (minimizing range), we need a negative feedback loop. If the phase lag of the system is significant, the sign might need to be flipped or the phase of the demodulation signal adjusted.
4. Actuation:
   Sum the bias and the dither to form the command.

   $$\omega_{cmd} = \hat{\omega}_{bias} + a \sin(\omega_0 t)$$

5.3 Tuning for the Crazyflie

Successful ESC depends heavily on parameter tuning relative to the drone's physics and the 20ms loop rate.

- **Perturbation Frequency (ω_0):** It must be slow enough for the drone's yaw dynamics to track but fast enough to provide frequent gradient updates. The Crazyflie is agile; a frequency of $1.0 - 2.0$ Hz (2π to 4π rad/s) is appropriate.
- **Amplitude (a):** Needs to be large enough to produce a range change detectable above the noise floor (10cm). If $v=0.3$ m/s, a yaw wiggle of $\pm 20^\circ$ is usually sufficient.
- **High-Pass Cutoff (ω_h):** Should be lower than ω_0. Typically $\omega_h \approx \omega_0 / 2$.
- **Latency Compensation:** Due to the PC-radio lag, the measured range $y(t)$ corresponds to the position at $t - \tau_{delay}$. The demodulation signal should be $\sin(\omega_0(t - \tau_{delay}))$. Empirically, tuning a phase offset phi in sin(omega * t + phi) maximizes the convergence speed.

Robustness Insight: ESC is essentially a "lock-in amplifier" in software. It is remarkably robust to noise because the correlation process filters out any noise that is not at the specific dither frequency ω_0. This makes it ideal for the noisy UWB environment.[8]

6. Algorithmic Strategy III: Temporal Trilateration (Synthetic Aperture)

This strategy adopts a geometric rather than control-theoretic perspective. It exploits the memory of the system. While the drone cannot see bearing instantaneously, it can remember where it *was* and what the range *was*, effectively creating a "Synthetic Aperture" array of virtual sensors along its flight path.

6.1 Concept and Geometry

As the drone moves from $\mathbf{p}_1$ to $\mathbf{p}_2$, it collects ranges r_1 and r_2. The anchor must lie at the intersection of Circle 1 (centered at $\mathbf{p}_1$, radius r_1) and Circle 2 (centered at $\mathbf{p}_2$, radius r_2).
Two circles intersect at two points (ambiguity). A third measurement at $\mathbf{p}_3$ (if not collinear with $\mathbf{p}_1, \mathbf{p}_2$) resolves this ambiguity to a single point.21
The critical missing link is the relative position $\Delta \mathbf{p} = \mathbf{p}_2 - \mathbf{p}_1$.

- **With Flow Deck:** The drone provides $\Delta x, \Delta y$ via optical flow integration.
- Without Flow Deck (User Scenario): We must rely on Dead Reckoning based on the velocity commands sent.

$$\mathbf{p}_{est}(t) = \int_0^t \mathbf{v}_{cmd}(\tau) d\tau$$

While dead reckoning drifts significantly over time due to drag, wind, and battery voltage sag, for a short window (e.g., 2-3 seconds), it is sufficiently accurate to estimate the local geometry of the anchor.

6.2 Least-Squares Estimation (Python Implementation)

Instead of analytical circle intersection (which is noisy), a non-linear least squares optimization is preferred. The Python scipy.optimize module is perfectly suited for this.

Algorithm Logic:

1. **Circular Buffer:** Maintain a rolling history of the last N seconds of data: $\{ (\mathbf{v}_i, t_i, r_i) \}$.
2. **Path Reconstruction:** Reconstruct the relative trajectory $\mathbf{p}_{rel}(t)$ from the velocity history, setting the current position as $(0,0)$.
3. Optimization: Define the cost function (residuals):

   $$E(x_a, y_a) = \sum_{i=1}^{M} \left(\sqrt{(x_{rel,i} - x_a)^2 + (y_{rel,i} - y_a)^2} - r_i \right)^2$$

   Solve for (x_a, y_a) that minimizes E.
4. Guidance: Calculate the bearing to the estimated (x_a, y_a) relative to the current heading.

   $$\psi_{target} = \text{atan2}(y_a, x_a)$$

   Apply a Proportional (P) controller to steer the yaw toward ψ_{target}.

6.3 Suitability Analysis

This method provides the smoothest path (straight line) once the estimate converges. However, it suffers from a "cold start" problem. If the drone is stationary or moving directly towards the anchor, the matrix becomes singular (lack of observability), and the estimator is unstable.

- **Requirement:** The drone *must* perform an initialization maneuver (e.g., a "figure-8" or a sideways slide) to populate the buffer with geometrically diverse data points before the estimator becomes reliable.[24]
- **Computational Load:** Running a non-linear least squares solver at 50 Hz in Python is feasible on a modern PC, but care must be taken not to block the communication thread.

7. System Implementation Guide

This section translates the theoretical strategies into specific implementation steps using the cflib Python ecosystem.

7.1 Data Link Configuration (LogConfig)

To achieve the 20ms (50 Hz) requirement, the logging configuration must be precise.

Python

```
# Python cflib snippet
from cflib.crazyflie.log import LogConfig

# Setup logging for range. 'ranging.distance0' assumes anchor ID 0.
# The variable type is float (meters) in recent firmware.
lg_range = LogConfig(name='Range', period_in_ms=20)
lg_range.add_variable('ranging.distance0', 'float')

# Critical: Use a callback to handle data asynchronously
def range_callback(timestamp, data, logconf):
    range_val = data['ranging.distance0']
    # Push to thread-safe queue or update global state
    shared_state.update_range(range_val, timestamp)

cf.log.add_config(lg_range)
lg_range.data_received_cb.add_callback(range_callback)
```

- **Warning:** If the firmware ranging rate (which depends on TWR scheduling) is slower than the log rate, you will receive duplicate values. The Python script must check if current_val!= prev_val to avoid feeding zero-derivative data into the algorithms (especially critical for ESC).[26]

7.2 Sending Control Commands

The MotionCommander class is user-friendly but blocking. For a 50Hz control loop, the lower-level Commander or non-blocking usage is required.
The recommended command for "Loco-only" flight (no Flow deck) is essentially a "guided hover" where we rely on the internal stabilizer to handle attitude, while we input generalized velocity/thrust targets.
However, without Flow/Lighthouse, send_velocity_world_setpoint or send_hover_setpoint might behave unpredictably because the drone lacks an onboard velocity estimator (it usually fuses Flow/UWB for this).

- **If Custom Firmware provides velocity estimation via UWB (e.g., EKF):** Use send_hover_setpoint(vx, vy, yaw_rate, z).
- **If Pure Range Only (No onboard velocity est):** You must control **Attitude** (Roll/Pitch angles) directly.
  - Forward flight $\approx$ Pitch -5 degrees.
  - This makes "Strategy III (Trilateration)" very hard because velocity is unknown.
  - **Recommendation:** Use Strategy II (ESC) and map the "Forward Velocity" parameter to a fixed "Pitch Angle" (e.g., -2 degrees) and "Yaw Rate" to the ESC output.

7.3 Latency Handling Architecture

The control loop should run in a dedicated thread or a non-blocking while loop separate from the communication callbacks.

Proposed Threading Model:

1. **Comms Thread (cflib):** Receives packets, timestamps them, puts them in a variable latest_range.
2. **Control Thread (50 Hz):**
   - Wake up every 20ms.
   - Read latest_range.
   - Apply **Median Filter** (Window=5).
   - Run **ESC Algorithm** (Update Integrator).
   - Calculate yaw_cmd and pitch_cmd.
   - Send packet: cf.commander.send_setpoint(roll=0, pitch=pitch_cmd, yawrate=yaw_cmd, thrust=thrust_cmd).

8. Advanced Nuances and Second-Order Insights

8.1 The "Ambiguity Circle" and Initial Conditions

When the mission starts, the drone is at distance R from the anchor. It could be anywhere on the circle of radius R.

- **Insight:** If the ESC dither is too small, or the drone starts by flying perfectly tangent to the circle, the gradient $\dot{r}$ will be zero. The integrator won't wind up, and the drone will orbit forever.
- **Mitigation:** The initial condition should include a "Kick": hard-code a turn for the first 2 seconds to force a gradient change, or set a non-zero initial value for the ESC integrator.[27]

8.2 Ground Effect and Near-Field Instability

As the drone approaches the anchor, two physical effects emerge:

1. **Near-Field UWB:** When distance < 20cm, the UWB antenna physics (near-field

reactive region) and signal saturation can cause the range reading to become highly non-linear or noisy.[28]

2. **Singularity:** At $r=0$, the bearing is undefined. $\dot{r}$ changes sign infinitely fast if the drone overshoots.

- **Safety Rule:** The algorithm must have a "Terminal State." If range < 0.5m, switch to HOVER or LAND. Do not try to fly to $r=0$.

8.3 The Multi-Drone Interference Factor

While the user asks about a single drone, UWB is a shared medium. If the user later adds a second drone, the TWR success rate will drop, leading to effectively lower update rates (e.g., 20Hz instead of 50Hz). The Python filters (Median/LPF) must be defined in *time domain* (based on timestamps), not *sample domain* (index), to handle variable data rates gracefully.[30]

9. Conclusion

Navigating a Crazyflie quadrotor to a single radio anchor using only range measurements is a challenging but solvable problem that requires shifting from instantaneous state feedback to temporal active sensing.

While **Reactive Gradient Following** offers a simple, code-light solution suitable for initial testing, it results in inefficient, oscillatory flight paths. **Temporal Trilateration** offers precision but is brittle without velocity feedback (Flow deck) and computationally heavier.

Therefore, **Extremum Seeking Control (ESC)** is identified as the optimal strategy for the specified hardware (Loco deck, single anchor, Python control). It leverages the drone's agility to perform the necessary gradient probing without requiring a complex internal map or odometry. By implementing a 1-2 Hz sinusoidal yaw perturbation and demodulating the range response, the Crazyflie can robustly lock onto the "signal source" (anchor) and home in effectively.

Success in implementation will depend critically on the signal conditioning pipeline—specifically the use of median filters to reject UWB outliers—and the proper tuning of the dither frequency to avoid resonance with the flight dynamics. The separation of high-frequency attitude stabilization (firmware) and low-frequency guidance (Python) allows this networked control system to function reliably despite the inherent latencies of the radio link.

Table 2: Comparative Summary of Strategies

Feature	Reactive (Braitenberg)	Extremum Seeking (ESC)	Temporal Trilateration

Logic	Bang-Bang (If/Else)	Continuous Control (Adaptive)	Geometric / Optimization
Input Req	$\text{sign}(\dot{r}) $	$\dot{r}$ + Time	Range History + Velocity Est
Motion	Zig-Zag / Spiral	Smooth Sinusoidal Weave	Straight (after init)
Computational Cost	Very Low	Low	High (Least Squares)
Robustness	High (immune to bias)	High (immune to noise)	Low (sensitive to drift)
Recommendation	**Initial Test**	**Primary Solution**	Only with Flow Deck

Works cited

1. Homing Guidance Using Spatially Quantized Signals - CORE, accessed on December 9, 2025, https://core.ac.uk/download/pdf/148428566.pdf
2. (PDF) Range-Only Bearing Estimator for Localization and Mapping, accessed on December 9, 2025, https://www.researchgate.net/publication/370071218_Range-Only_Bearing_Estimator_for_Localization_and_Mapping
3. Active Range-Only Beacon Localization for AUV Homing - ResearchGate, accessed on December 9, 2025, https://www.researchgate.net/publication/279848126_Active_Range-Only_Beacon_Localization_for_AUV_Homing
4. Source seeking with a nonholonomic unicycle without position measurements and with tuning of angular velocity part I: Stability analysis | Request PDF - ResearchGate, accessed on December 9, 2025, https://www.researchgate.net/publication/224303673_Source_seeking_with_a_nonholonomic_unicycle_without_position_measurements_and_with_tuning_of_angular_velocity_part_I_Stability_analysis
5. The Loco positioning system - Bitcraze, accessed on December 9, 2025, https://www.bitcraze.io/2022/06/the-loco-positioning-system/
6. Learning-based Bias Correction for Accurate Ultra-wideband Localization of a Crazyflie, accessed on December 9, 2025, https://www.bitcraze.io/2020/04/learning-based-bias-correction-for-accurate-ultra-wideband-localization-of-a-crazyflie/

7. Model-free source seeking of exponentially convergent unicycle: theoretical and robotic experimental results | Request PDF - ResearchGate, accessed on December 9, 2025, https://www.researchgate.net/publication/397232075_Model-free_source_seeking_of_exponentially_convergent_unicycle_theoretical_and_robotic_experimental_results
8. Stochastic source seeking for nonholonomic unicycle - Miroslav Krstic, accessed on December 9, 2025, https://flyingv.ucsd.edu/papers/PDF/135.pdf
9. Nonlocal Nonholonomic Source Seeking Despite Local Extrema arXiv:2303.16027v1 [math.OC] 28 Mar 2023, accessed on December 9, 2025, https://arxiv.org/pdf/2303.16027
10. Exploring Data Anomalies: Rejecting Outliers with Python | by Prashant S | Medium, accessed on December 9, 2025, https://medium.com/@saraswatp/exploring-data-anomalies-rejecting-outliers-with-python-660b1ed6bca6
11. (PDF) UWB-based 3D Localization using Least Squares Trilateration with Combination of Median Filter and Kalman Filter - ResearchGate, accessed on December 9, 2025, https://www.researchgate.net/publication/391100908_UWB-based_3D_Localization_using_Least_Squares_Trilateration_with_Combination_of_Median_Filter_and_Kalman_Filter
12. Extremum Seeking Control - MATLAB & Simulink - MathWorks, accessed on December 9, 2025, https://www.mathworks.com/help/slcontrol/ug/extremum-seeking-control.html
13. Category: Testing - Bitcraze, accessed on December 9, 2025, https://www.bitcraze.io/category/crazyflie/testing/
14. Latency measurement between PC and Crazyflie - Bitcraze Forums, accessed on December 9, 2025, https://forum.bitcraze.io/viewtopic.php?t=4691
15. A Range-Based Algorithm for Autonomous Navigation of an Aerial Drone to Approach and Follow a Herd of Cattle - PMC - NIH, accessed on December 9, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC8588052/
16. US11899409B2 - Extremum seeking control system and a method for controlling a system - Google Patents, accessed on December 9, 2025, https://patents.google.com/patent/US11899409B2/ja
17. Nonlocal Nonholonomic Source Seeking Despite Local Extrema - Miroslav Krstic, accessed on December 9, 2025, https://flyingv.ucsd.edu/papers/PDF/435.pdf
18. An Introduction to Extremum-Seeking Control - A. Skafte, accessed on December 9, 2025, https://alexanderskafte.com/esc.pdf
19. Extremum Seeking - Miroslav Krstic, accessed on December 9, 2025, https://flyingv.ucsd.edu/krstic/talks/talks-files/extremum-seeking-DISC12.pdf
20. Extremum Seeking Control With an Adaptive Gain Based on Gradient Estimation Error | Request PDF - ResearchGate, accessed on December 9, 2025, https://www.researchgate.net/publication/360433643_Extremum_Seeking_Control_With_an_Adaptive_Gain_Based_on_Gradient_Estimation_Error
21. Position of Two Circles - Analytic Geometry - Mathematics - Grade 12 - High

School - Nakafa, accessed on December 9, 2025, https://nakafa.com/en/subject/high-school/12/mathematics/analytic-geometry/position-of-two-circles
22. Calculate the intersection points of two Circles - RAW, accessed on December 9, 2025, https://raw.org/math/calculate-the-intersection-points-of-two-circles/
23. Find the points where two circles intersect - Applied Mathematics Consulting, accessed on December 9, 2025, https://www.johndcook.com/blog/2023/08/27/intersect-circles/
24. Target Motion Analysis Using Single Sensor Bearings-Only Measurements - ResearchGate, accessed on December 9, 2025, https://www.researchgate.net/publication/224610031_Target_Motion_Analysis_Using_Single_Sensor_Bearings-Only_Measurements
25. State estimation in range coordinate using range-only measurements - IEEE Xplore, accessed on December 9, 2025, https://ieeexplore.ieee.org/iel7/5971804/9813508/09813536.pdf
26. The Crazyflie Python API explanation - Bitcraze, accessed on December 9, 2025, https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/user-guides/python_api/
27. Analysis of an Extremum Seeking Controller Under Bounded Disturbance - NSF PAR, accessed on December 9, 2025, https://par.nsf.gov/servlets/purl/10330162
28. Datasheet Loco positioning node - Rev 1 - Bitcraze, accessed on December 9, 2025, https://www.bitcraze.io/documentation/hardware/loco_node/loco_node-datasheet.pdf
29. Measurements of accuracy and precision of the Loco Positioning system - Bitcraze, accessed on December 9, 2025, https://www.bitcraze.io/documentation/system/positioning/accuracy-loco/
30. Category: Crazyflie | Bitcraze, accessed on December 9, 2025, https://www.bitcraze.io/category/crazyflie/page/13/
31. Bitcraze - GitHub, accessed on December 9, 2025, https://github.com/bitcraze